CS 423
Operating System Design: Processes and CPU Virtualization
Aug 27

Ram Alagappan

Figures and slides ack: Prof. Shivaram Venkataraman (Wisconsin)

AGENDA / OUTCOMES

3 pieces: Virtualization, Concurrency, and Persistence

Abstraction

What is a Process? What is its lifecycle?

Mechanism

How does process interact with the OS?

How does the OS switch between processes?

What we won't cover here, but you should read up:

Ch4+5: process-related data structures. fork() & exec().

ABSTRACTION: PROCESS

PROGRAM VS PROCESS

```
#include <stdio.h>
#include <stdlib.h>
#include "common.h"

int main(int argc, char *argv[]) {
    char *str = argv[1];
    int i = 0;
    while (1) {
      printf("%s\n",str);
      i++;
    }
    return 0;
}
```

Program

Process

WHAT IS A PROCESS?

Stream of executing instructions + associated “context”

```
pushq  %rbp
movq   %rsp, %rbp
subq   $32, %rsp
movl   $0, -4(%rbp)
movl   %edi, -8(%rbp)
movq   %rsi, -16(%rbp)
cmpl   $2, -8(%rbp)
je LBB0_2
```

Registers
Memory addrs

File descriptors

WHAT IS A PROCESS?

Stream of executing instructions + associated “context”

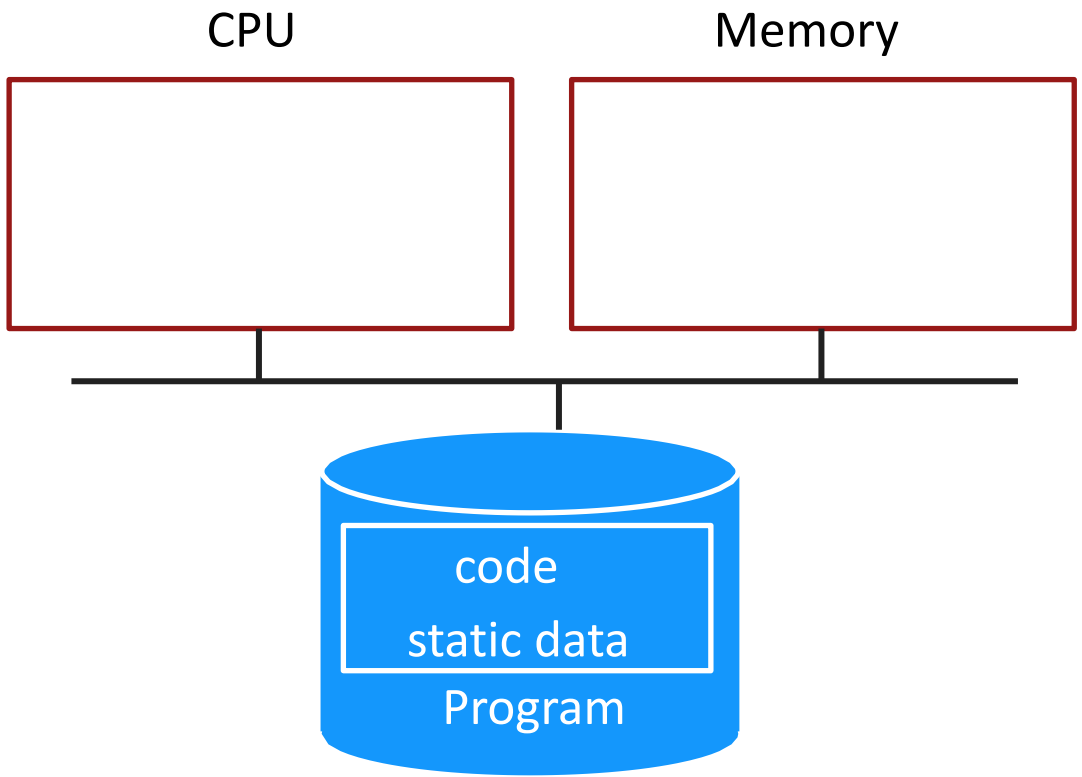
PC: program counter
aka IP
SP: stack pointer
FP: frame pointer

```
pushq  %rbp
movq   %rsp, %rbp
subq   $32, %rsp
movl   $0, -4(%rbp)
movl   %edi, -8(%rbp)
movq   %rsi, -16(%rbp)
cmpl   $2, -8(%rbp)
je LBB0_2
```

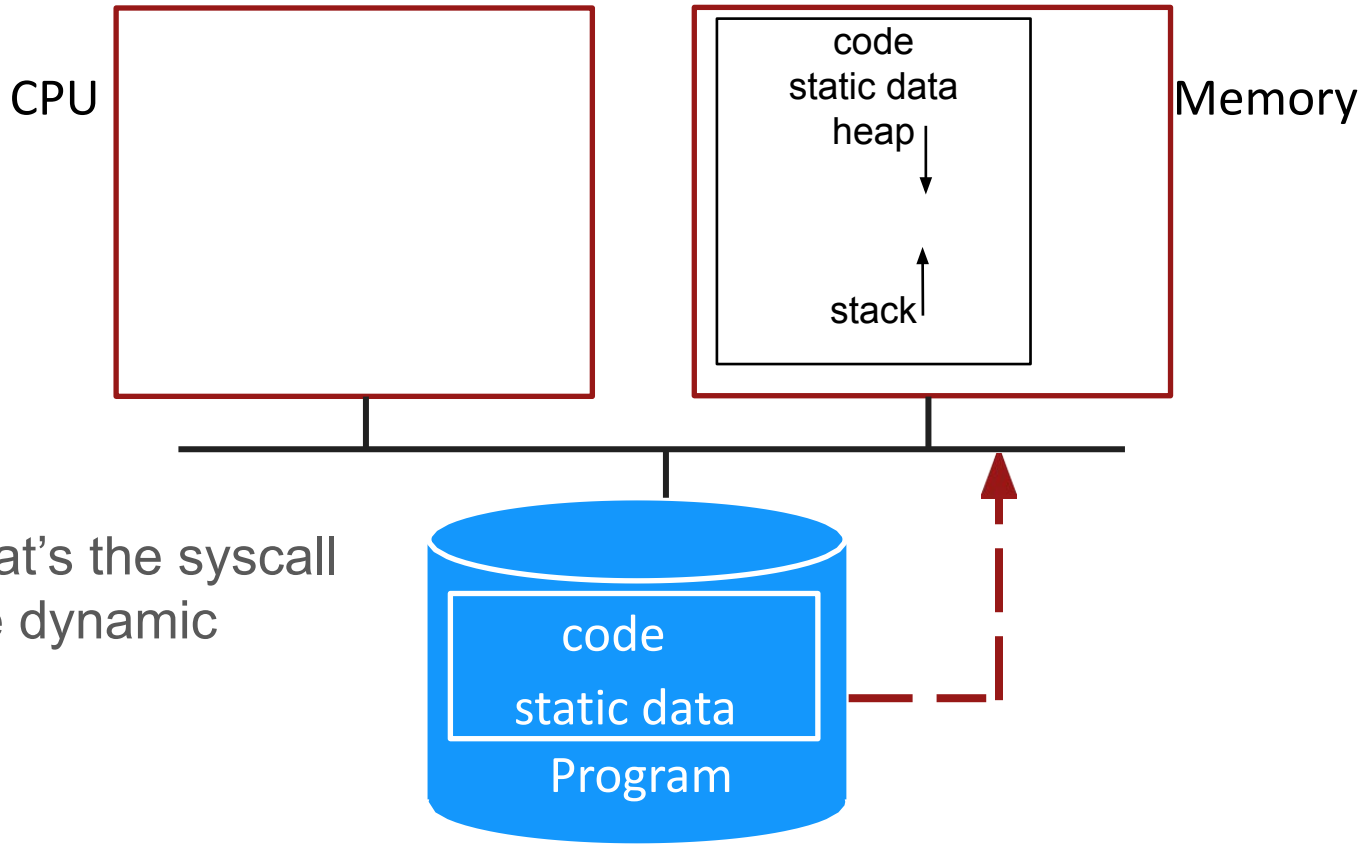
Registers
Memory addrs

File descriptors

PROCESS CREATION

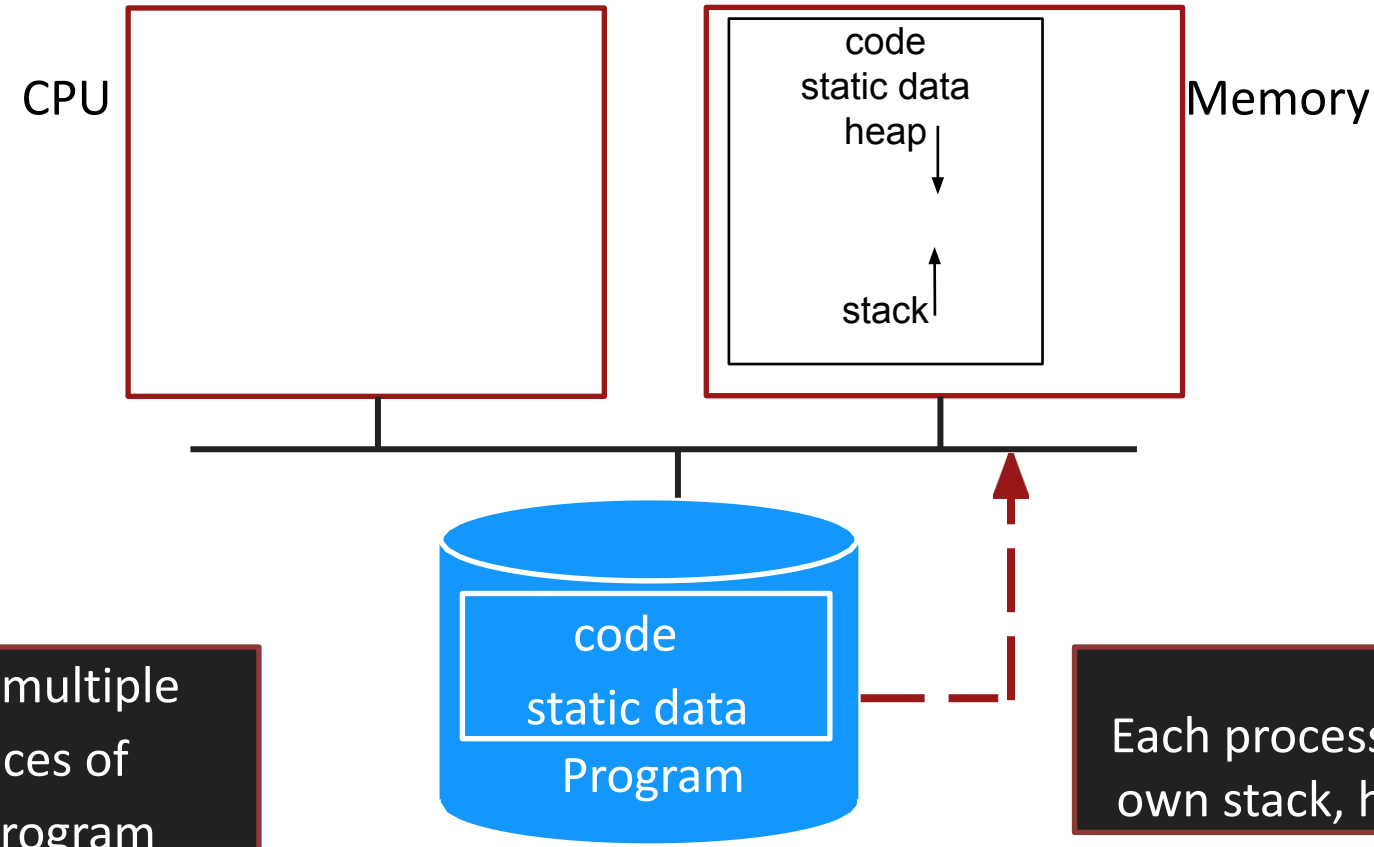


PROCESS CREATION



Aside: what's the syscall to allocate dynamic memory?

PROCESS CREATION



Can run multiple instances of same program

Each process has its own stack, heap etc.

PROCESS VS THREAD

Threads: “Lightweight process”

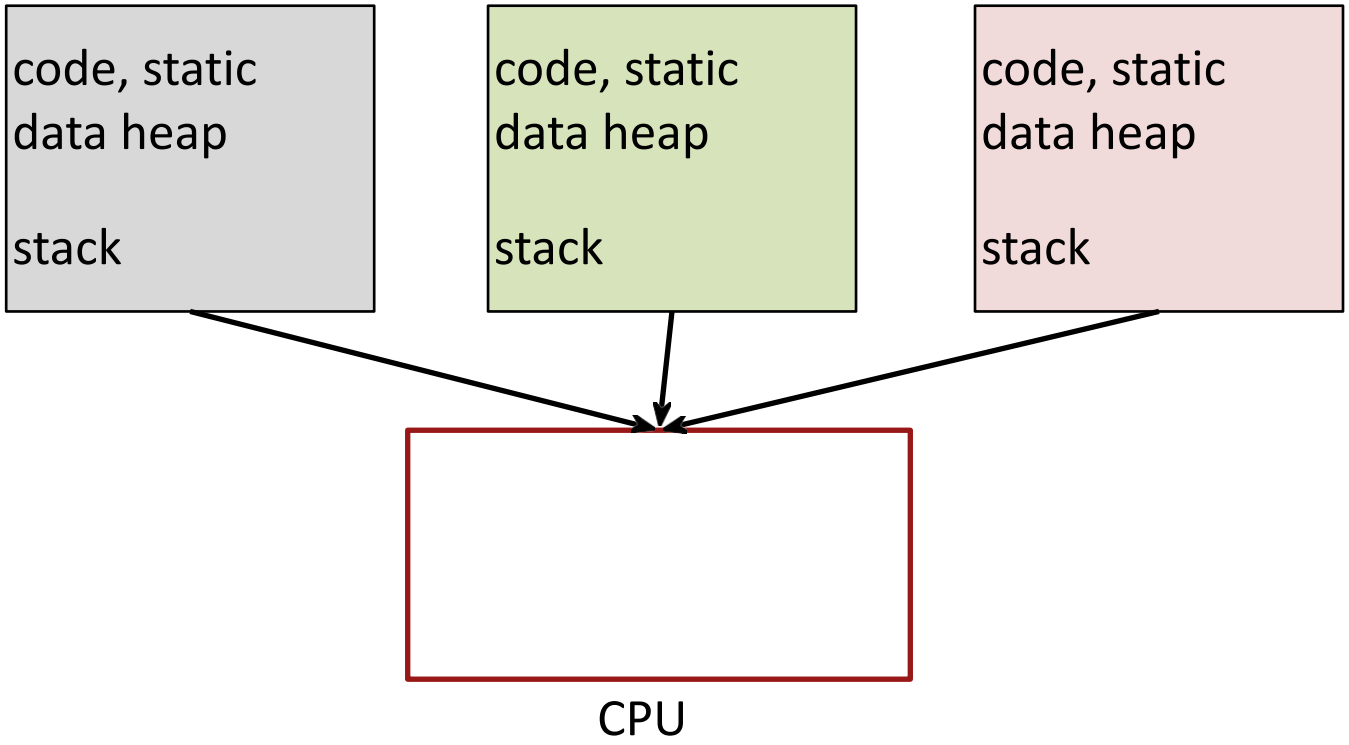
Can have multiple threads within a single process

Shared across threads (True or False):

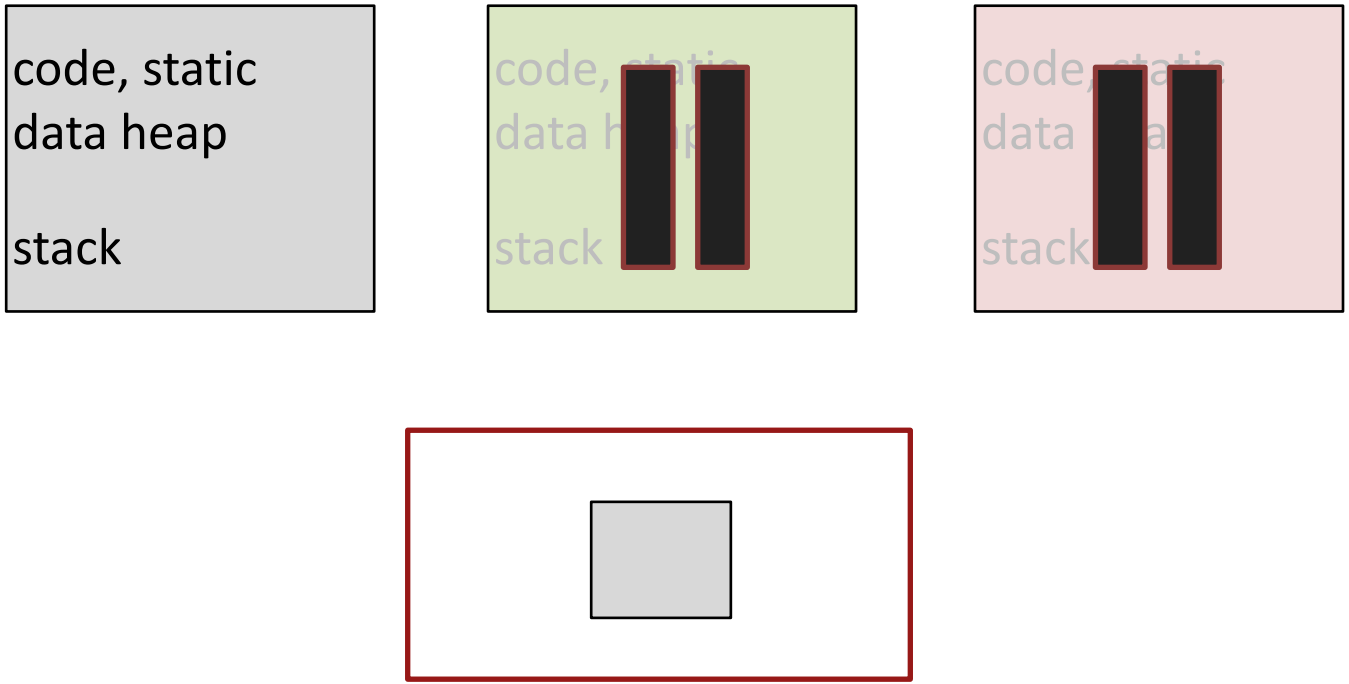
code? globals? heap? stack? file descriptors? registers?

SHARING THE CPU

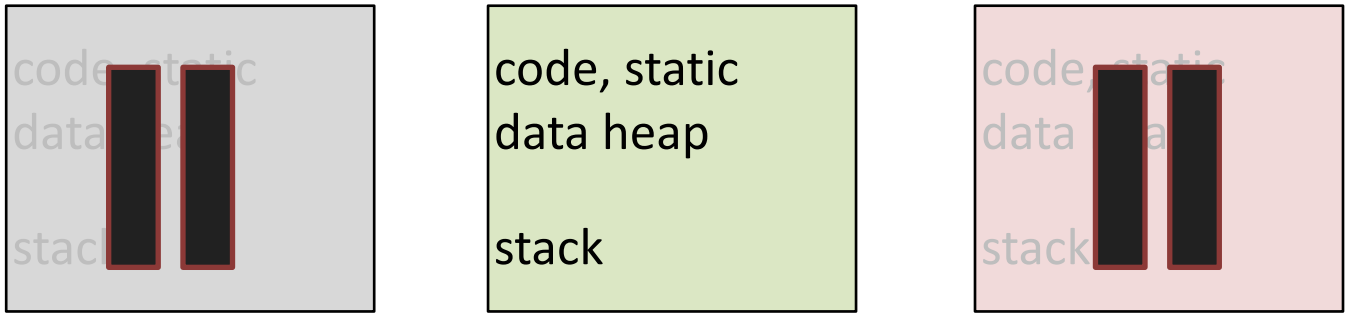
SHARING CPU


code, static
data heap
stack
code, static
data heap
stack
code, static
data heap
stack
CPU

TIME SHARING



TIME SHARING

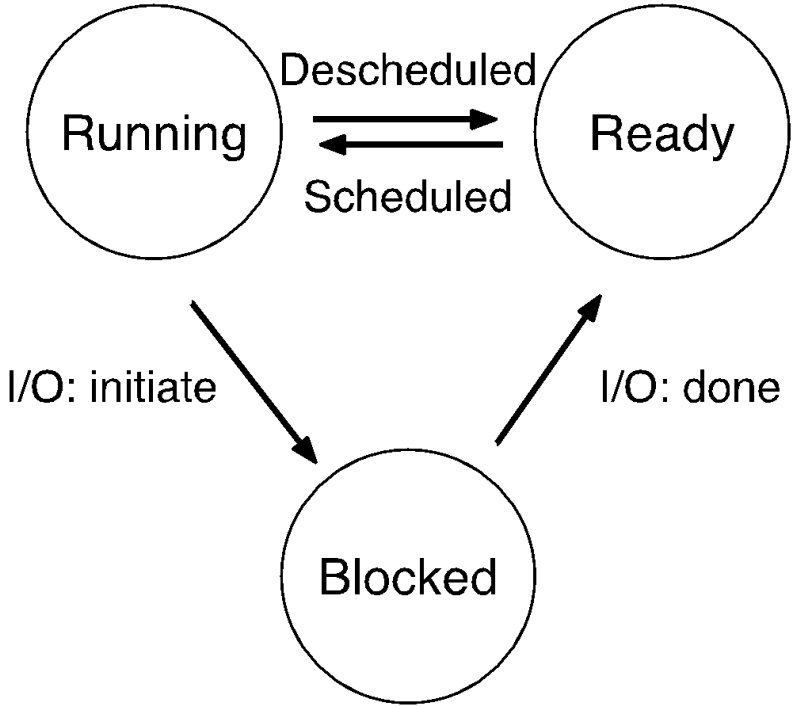


WHAT TO DO WITH PROCESSES THAT ARE NOT RUNNING ?

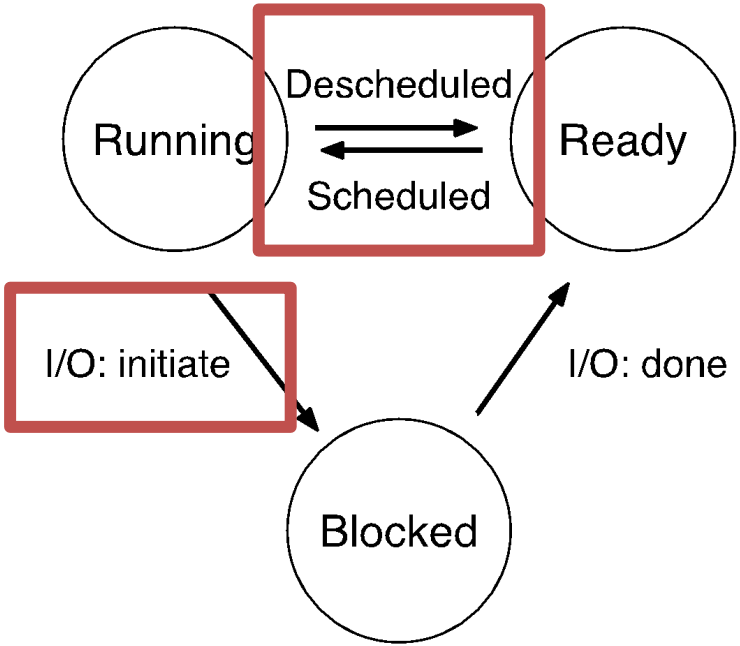
OS Scheduler

- Save context (aka state) when pausing process
- Restore context on resumption

STATE TRANSITIONS



STATE TRANSITIONS



Example

Process 0
 io
 io
 cpu (1 unit)

Process 1
 cpu (4 units)
  io
  io

Each IO takes 4 time units

Time	PID: 0	PID: 1
1	RUNNING io	READY
2		
3		
4		
5		
6		
7		
8		

Example

Process 0
 io
 io
 cpu (1 unit)

Process 1
 cpu (4 units)
  io
  io

Each IO takes 4 time units

Time	PID: 0	PID: 1
1	RUNNING io	READY
2	BLOCKED	RUNNING cpu
3	BLOCKED	RUNNING cpu
4	BLOCKED	RUNNING cpu
5	BLOCKED	RUNNING cpu
6	READY	RUNNING io
7	RUNNING io	BLOCKED
8	BLOCKED	BLOCKED

CPU SHARING

Policy goals

- Virtualize CPU resource using processes
- Higher CPU utilization? Fairness?

Mechanism goals

- Efficiency: Sharing should not add much overhead
- Control: OS should be able to intervene when required

Today, we're focused on only mechanism

EFFICIENT EXECUTION

Answer: Direct Execution

What does “run directly on the CPU” mean?

User process runs directly on the CPU (no OS interposition)

Create process and transfer control to main()

Problems with DE?

Problems with DE?

Problems with DE:

Restricted ops: What if the process wants to do something restricted like allocate resources, access IO devices, etc?

How to switch processes: What if the process runs “forever”? e.g., a buggy loop, a malicious program that wants to take all CPU time

General solution: Limited Direct Execution (LDE)

PROBLEM1: RESTRICTED OPS

How can we ensure user process can't harm others?

Solution: privilege levels supported by hardware

- CPU: has a mode bit
- User process runs in user mode (restricted mode)
- OS runs in kernel mode (unrestricted)

How can a process access restricted ops?

- system call: function call implemented by OS

SYSTEM CALL

Syscall

Trap instruction :

Changes to unrestricted or kernel mode

What is it in x86? INT

Ret-from-trap instruction:

Return from kernel to user mode

What is it in x86? IRET

C Libraries usually hide these instructions and give a nicer interface like read()/write()

Syscall

Must save callee's instruction pointer, stack pointer, and relevant registers to resume after syscall

Where are these saved?

Kernel stack: every process has its own kernel stack (more precisely every thread has its own kernel stack)

Operating System | Hardware | Program

Process A

Run main() ...
Call system call
trap into OS

move to kernel mode
save regs(A) to k-stack(A)
jump to trap handler

Handle the trap
Do work of syscall
return-from-trap

Restore regs (from kstack)
move to user mode
jump to PC past trap instruction

Syscall

How does the hardware know where to jump (i.e., trap handler location)?

Solution: trap table & system call table
INT instruction tells trap handler to consult syscall table

At boot OS “configures” hardware with trap handler locations

On trap, hardware simply jumps to this location

OS knows this is a syscall, uses syscall number to invoke particular syscall

read(fd, buf, count)

```
mov $3, %eax          ; 3 => SYS_read
mov  fd, %ebx         ; file descriptor
mov buf, %ecx
mov count, %edx

INT 0x80              ; invoke trap with 0x80
```

// Note: modern systems use SYSCALL instead of INT

```
#define __NR_exit 1
#define __NR_fork 2
#define __NR_read 3
#define __NR_write 4
#define __NR_open 5
#define __NR_close 6
#define __NR_waitpid 7
#define __NR_creat 8
#define __NR_link 9
#define __NR_unlink 10
```

user program
  |
  | int $0x80
  v
CPU looks up IDT[0x80]
  |
  v
switch to kernel privilege
  |
  v
syscall entry handler
  |
  v
inspect EAX → which syscall?
(read, write, ...)

SYSCALL SUMMARY

Separate user-mode from kernel mode for security

Syscall: call kernel mode functions

- Transfer from user-mode to kernel-mode (trap)
- Return from kernel-mode to user-mode (return-from-trap)

PROBLEM2: HOW TO TAKE CPU AWAY?

Policy

- To decide which process to schedule
- More next lecture

Mechanism

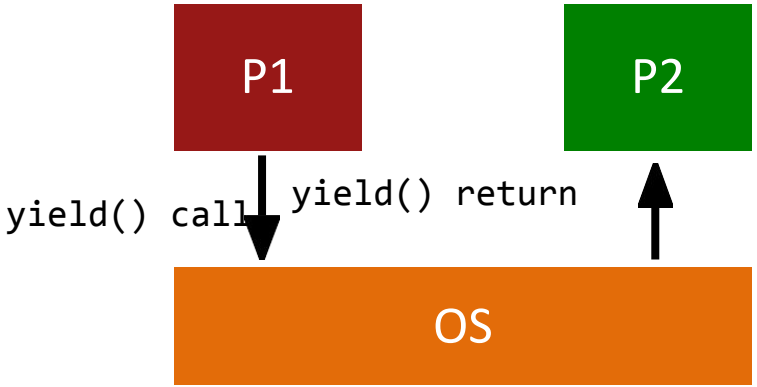
- Fast switch between processes
- Low-level code that implements the switch

Separation of policy and mechanism: Recurring theme in OS

HOW CAN OS GET CONTROL?

Option 1: Cooperative Multi-tasking: Trust process to relinquish CPU

- Examples: System call, page fault (accessed page not in main memory), or error (illegal instruction or divide by zero)
- Provide special `yield()` system call



PROBLEMS WITH COOPERATIVE ?

Disadvantage: Processes can misbehave

By avoiding all traps and performing no I/O, can take over entire machine

Only solution: Reboot!

Not performed in modern operating systems

OS alone cannot do much here…
So, what's the help we need from the hardware?

TIMER-BASED INTERRUPTS

Option 2: Timer-based Multi-tasking
Guarantees OS control within a deterministic time period

Enter OS by using a periodic “alarm clock”

Hardware generates timer interrupt (CPU or separate chip)

Example: Every 10ms

User code cannot turn off time, OS can in some cases

Operating System

Hardware

Program

Process A

timer interrupt

move to kernel mode

save regs(A) to k-stack(A)

jump to trap handler

Operating System | Hardware | Program

Process A

timer interrupt
move to kernel mode
save regs(A) to k-stack(A)
jump to trap handler

Handle the trap
Call switch() routine
save kernel regs(A) to proc-struct(A)
restore kernel regs(B) from proc-struct(B)
switch to k-stack(B)
return-from-trap (into B)

Operating System | Hardware | Program

Process A

timer interrupt

move to kernel mode

save regs(A) to k-stack(A)

jump to trap handler

Handle the trap

Call switch() routine

save kernel regs(A) to proc-struct(A)

restore kernel regs(B) from proc-struct(B) // when was this saved?

switch to k-stack(B) // this is the key point

return-from-trap (into B)

restore regs(B) from k-stack(B) move to user mode

jump to B's IP

Process B

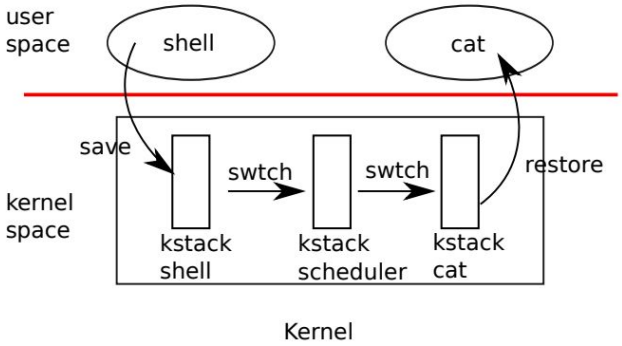
Real xv6 Example (if you are interested)

```
void
sched(void)
{
  int intena;
  struct proc *p = myproc();

  if(!holding(&ptable.lock))
    panic("sched ptable.lock");
  if(mycpu()->ncli != 1)
    panic("sched locks");
  if(p->state == RUNNING)
    panic("sched running");
  if(readeflags()&FL_IF)
    panic("sched interruptible");
  intena = mycpu()->intena;
  swtch(&p->context, mycpu()->scheduler);
  mycpu()->intena = intena;
}
```

- Scheduler switches to user process in “scheduler” function
- User process switches to scheduler thread in the “sched” function

```
void
scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();
  c->proc = 0;

  for(;;){
    // Enable interrupts on this processor.
    sti();

    // Loop over process table looking for process to run.
    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->state != RUNNABLE)
        continue;

      // Switch to chosen process.  It is the process's job
      // to release ptable.lock and then reacquire it
      // before jumping back to us.
      c->proc = p;
      switchuvm(p);
      p->state = RUNNING;

      swtch(&(c->scheduler), p->context);
      switchkvm();

      // Process is done running for now.
      // It should have changed its p->state before coming back.
      c->proc = 0;
    }
    release(&ptable.lock);

  }
}
```

xv6 Example (If you are interested)



There is a separate per-cpu scheduler thread in xv6 that makes the policy decision

If you are interested: take a look at

https://github.com/mit-pdos/xv6-public

proc.c: “sched” function where the old user process switches into the scheduler thread

proc.c: “scheduler” function where the scheduler switches into the new user process

SUMMARY

Process: Abstraction to virtualize CPU resource

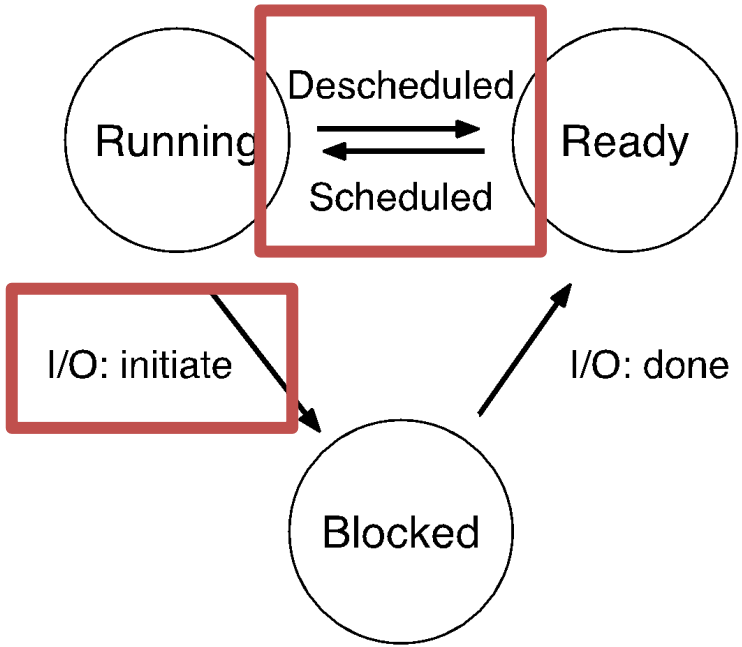
Time-sharing in OS to switch between processes

Key aspects of Mechanism

- System calls to access restricted ops
- Time-sharing: context-switch via timer interrupt

What we didn’t cover here, but you should read up:

- Ch4+5: process-related data structures. fork() & exec().



POLICY ?
Next lecture!